# Simulacrum

A Modular Computational Mechanics Library

Numerical Methods for PDEs · Scientific Machine Learning · Uncertainty Quantification

Ioannis Kalogeris

Version 0.1.0

February 2026

Python orchestration with optional Rust/C++ acceleration

MIT License

# Contents

# Chapter 1

# Introduction

## 1.1 Overview

Simulacrum is a modular computational mechanics library designed for research in numerical methods for partial differential equations (PDEs), scientific machine learning, and uncertainty quantification. The library follows a *Python-first* philosophy: all orchestration happens in Python using NumPy and SciPy, with optional compiled backends in Rust and C++ for performance-critical loops.

## 1.2 Design Principles

1. **Python first.** All orchestration in Python. NumPy/SciPy for numerics.
2. **Swappable discretisations.** FEM, FDM, FVM, and SPH share the same `Domain` → `LinearSystem` → `Solver` pipeline.
3. **Library vs. projects.** Reusable components live in `src/simulacrum/`; research workflows live in `projects/`.
4. **Compiled backends optional.** Rust (via maturin/PyO3) and C++ (via pybind11/Eigen) for hot loops. The library works without them.
5. **Meshing is external.** Use gmsh or meshio for mesh generation and import.
6. **VTK output.** Write `.vtu` files and visualise in ParaView or interactively with PyVista.

## 1.3 Installation

Install the core library in editable (development) mode:

```
cd ~/Desktop/pythonCodes/SImulacrum
pip install -e ".[dev]"
```

Install with optional dependency groups as needed:

```
pip install -e ".[jax,dev]"     # JAX + Equinox (for NSF project)
pip install -e ".[ml,dev]"      # PyTorch (for ML subpackage)
pip install -e ".[all]"         # Everything
```

# Chapter 2

# Project Structure

## 2.1 Top-Level Layout

```
Simulacrum/
    pyproject.toml          # Package config and dependencies
    README.md
    src/simulacrum/         # The library (reusable)
    projects/               # Research projects (consumers)
    tests/                  # Unit tests
    examples/               # Short demos
    benchmarks/             # Performance benchmarks
    docs/                   # Documentation source
```

- **src/simulacrum/** contains all reusable library code. Anything here should be general enough that multiple projects can import it.
- **projects/** contains full research workflows—each project has its own config, training scripts, data pipelines, and checkpoints.
- **examples/** contains short, self-contained scripts demonstrating library usage (e.g. `poisson_2d.py`).
- **tests/** contains pytest unit tests for the library modules.

## 2.2 Library Subpackages

```
src/simulacrum/
    core/               # Domain, Field, DOF, logging
    mesh/               # Mesh generation and import
    discretization/     # FEM, FDM, FVM, SPH
    materials/          # Constitutive models
    solvers/            # Linear and nonlinear solvers
    boundary/           # Boundary condition handling
    geometry/           # Shape generation, SDF, harmonic maps
    ml/                 # Machine learning
    uq/                 # Uncertainty quantification
    optimization/       # Optimisation methods
    io/                 # VTK/VTU output
    postprocessing/     # PyVista visualisation
    _backends/          # Rust and C++ acceleration
```

## 2.3 Projects

```
projects/
    nsf_domain_mapping/      # Neural Spline Flow: domain -> square
        models/              #   NSF architecture
        config.py            #   Hyperparameters
        dataset.py           #   Data generation pipeline
        Main_nsf.py          #   Full training/eval pipeline
        data/                #   Generated datasets
        checkpoints_nsf/     #   Trained models
```

# Chapter 3

# Core Simulation Modules

## 3.1 `core` — Domain, Field, and DOF Management

The `core` subpackage provides the foundational abstractions.

| Class/Function | Description |
|---|---|
| `Domain` | Represents a computational domain with mesh, fields, and boundary conditions |
| `Field` | Scalar or vector field defined on a mesh |
| `DOFManager` | Maps mesh nodes to degrees of freedom, handles numbering and constraints |
| `get_logger(name)` | Consistent logging across the library |

## 3.2 `mesh` — Mesh Generation and Import

| Class/Function | Description |
|---|---|
| `StructuredMesh` | Regular grid mesh generation (Quad4, Hex8) |
| `UnstructuredMesh` | Triangular/tetrahedral mesh via gmsh |
| `read_mesh(path)` | Import mesh files via meshio (.msh, .vtu, .vtk) |

## 3.3 `discretization` — Numerical Schemes

All discretisation methods inherit a common assembler interface that produces a sparse linear system $\mathbf{Ku} = \mathbf{f}$.

### 3.3.1 FEM — Finite Element Method

| Class/Function | Description |
|---|---|
| `FEMAssembler` | Assembles stiffness matrix and load vector |
| `Element` | Base class for element types (Tri3, Quad4, Tet4, Hex8) |
| `GaussQuadrature` | Numerical integration rules |
| `ShapeFunctions` | Shape function evaluation and gradients |

### 3.3.2 FDM — Finite Difference Method

| Class/Function | Description |
|---|---|
| `FDMAssembler` | Builds finite difference stencil matrix |
| `Stencil` | Configurable stencil patterns (3-point, 5-point, 9-point) |

## 3.4 `materials` — Constitutive Models

| Class | Description |
|---|---|
| `LinearElastic` | Isotropic linear elasticity (Young's modulus, Poisson's ratio) |
| `ThermalMaterial` | Thermal conductivity for heat transfer problems |
| `HyperelasticMaterial` | Base class for nonlinear materials (scaffold) |

## 3.5 `solvers` — Linear and Nonlinear Solvers

| Class | Description |
|---|---|
| `DirectSolver` | SciPy sparse direct solver (spsolve) |
| `ConjugateGradient` | CG iterative solver for SPD systems |
| `GMRES` | Generalised Minimal Residual for non-symmetric systems |
| `NewtonRaphson` | Newton–Raphson for nonlinear problems |

## 3.6 `boundary` — Boundary Conditions

| Class/Function | Description |
|---|---|
| `DirichletBC` | Prescribed displacement/temperature values |
| `NeumannBC` | Prescribed traction/flux on boundaries |
| `apply_bcs(K, f, bcs)` | Apply boundary conditions to the assembled system |

## 3.7 `io` and `postprocessing`

| Class/Function | Description |
|---|---|
| `write_vtu(path, mesh, fields)` | Write solution to VTU format |
| `plot_mesh(mesh)` | Interactive 3D mesh visualisation via PyVista |
| `plot_field(mesh, field)` | Scalar/vector field plots with colour maps |

# Chapter 4

# Geometry Subpackage

The `simulacrum.geometry` subpackage provides tools for generating parameterised 2D shapes, computing signed distance fields, and solving harmonic maps from arbitrary domains to the unit square.

## 4.1 `geometry.shapes` — Parameterised Closed Curves

All shape classes inherit from `ClosedCurve` and implement `__call__(t)` for evaluation at parameter $t \in [0, 2\pi]$.

| Class | Description |
|---|---|
| `ClosedCurve` | Abstract base class: `__call__(t)`, `sample(n)`, `contains(pts)` |
| `Ellipse` | Axis-aligned or rotated ellipse with semi-axes $a$, $b$ |
| `Superellipse` | Lamé curve: $\lvert x/a\rvert^n + \lvert y/b\rvert^n = 1$ |
| `FourierBlob` | Radial Fourier perturbation: $r(\theta) = r_0 + \sum a_k \cos(k\theta) + b_k \sin(k\theta)$ |
| `StarPolygon` | Star polygon with alternating inner/outer radii |
| `SmoothPolygon` | Polygon with rounded corners via Catmull–Rom splines |
| `ShapeGenerator` | Factory for generating random shapes with configurable distribution |

Each class provides a `random(rng)` class method for random instantiation.

Listing 4.1: Generating and sampling shapes

```
from simulacrum.geometry import ShapeGenerator, Ellipse

# Random shapes with custom distribution
gen = ShapeGenerator(seed=42, shape_distribution={"Ellipse": 0.5, "
    FourierBlob": 0.5})
shape = gen.generate()

# Specific shape
ellipse = Ellipse(center=(0, 0), a=1.2, b=0.7, theta=0.3)
points = ellipse.sample(500)  # (500, 2) array of boundary points
```

## 4.2 `geometry.sdf` — Signed Distance Fields

| Class/Function | Description |
|---|---|
| `SDFRenderer` | Batch renderer: shape → normalised SDF image |
| `render_shape_to_sdf(shape, ...)` | Compute SDF on a regular grid |
| `sample_points_in_shape(shape, n)` | Rejection sampling of interior points |
| `normalize_sdf(sdf, method)` | Normalise SDF to $[-1, 1]$ (tanh, clip, or linear) |

## 4.3 `geometry.harmonic_maps` — Harmonic Map Solver

Computes the ground-truth diffeomorphism $\varphi : \Omega \to [-1, 1]^2$ by solving two Laplace equations $\Delta u = 0$, $\Delta v = 0$ with boundary conditions mapping $\partial\Omega$ to the square boundary.

| Class/Function | Description |
|---|---|
| `HarmonicMapSolver` | Solve and cache harmonic maps for a given shape |
| `compute_harmonic_map(shape, ...)` | Full FD solve returning $(u, v)$ grids |
| `interpolate_harmonic_map(...)` | Bilinear interpolation at arbitrary query points |

# Chapter 5

# Machine Learning Subpackage

The `simulacrum.ml` subpackage provides neural network architectures, training utilities, and surrogate model wrappers built on PyTorch. Install with `pip install -e ".[ml]"`.

## 5.1 `ml.networks` — Neural Network Architectures

| Class | Description |
|---|---|
| `MLP` | Fully connected network with configurable depth, width, activation, dropout, and batch normalisation |
| `MeshGNN` | Graph neural network operating on mesh connectivity. Nodes = mesh nodes, edges = element connections |
| `UNet` | U-Net encoder–decoder for field-to-field regression on structured grids |

Listing 5.1: Creating an MLP

```python
from simulacrum.ml.networks import MLP

net = MLP(
    layer_sizes=[3, 128, 128, 1],
    activation="gelu",
    dropout=0.1,
)
print(net)  # MLP([3, 128, 128, 1], params=17,153)
```

## 5.2 `ml.training` — Training Utilities

| Class | Description |
|---|---|
| `Trainer` | Generic PyTorch training loop with early stopping, checkpointing, and LR scheduling |
| `MSELoss` | Standard mean squared error |
| `RelativeMSELoss` | Relative $L^2$ loss: $\|\hat{y} - y\|^2 / \|y\|^2$ |

| | |
|---|---|
| `PhysicsInformedLoss` | PDE residual loss using automatic differentiation |
| `WeightedLoss` | Weighted combination of multiple loss terms |

## 5.3 `ml.surrogates` — PDE Surrogate Models

| Class | Description |
|---|---|
| `PDESurrogate` | Wraps a trained network as a drop-in replacement for a PDE solver. Includes input/output scaling and batch prediction. |

# Chapter 6

# Uncertainty Quantification Subpackage

The `simulacrum.uq` subpackage provides sampling methods, Bayesian inference tools, polynomial chaos expansions, and Gaussian process emulators.

## 6.1 `uq.sampling` — Sampling Methods

| Function | Description |
|---|---|
| `monte_carlo(n, dim, bounds)` | Standard Monte Carlo sampling |
| `latin_hypercube(n, dim, bounds)` | Latin Hypercube Sampling for better space-filling |
| `sobol_sequence(n, dim, bounds)` | Sobol quasi-random low-discrepancy sequence |
| `importance_sampling(...)` | Importance sampling with normalised weights |

## 6.2 `uq.bayesian` — Bayesian Inference

| Class | Description |
|---|---|
| `MetropolisHastings` | Random-walk Metropolis–Hastings with configurable proposal |
| `AdaptiveMCMC` | Adaptive Metropolis with Haario-style covariance adaptation |
| `RealNVP` | Normalizing flow for variational inference (affine coupling layers) |
| `MCMCResult` | Container with samples, log-probs, thinning, and burn-in methods |

## 6.3 `uq.pce` — Polynomial Chaos Expansions

| Class | Description |
|---|---|

| | |
|---|---|
| `PolynomialChaosExpansion` | Non-intrusive PCE via least-squares. Supports Legendre (uniform) and Hermite (Gaussian) bases. Properties: `mean`, `variance`, `sobol_indices()`. |

## 6.4 `uq.gp` — Gaussian Process Emulation

| Class | Description |
|---|---|
| `GaussianProcessEmulator` | Scalar-output GP with RBF/ARD kernel. Returns `GPPrediction` with mean, std, and variance. |

# Chapter 7

# Optimisation Subpackage

The `simulacrum.optimization` subpackage provides gradient-based optimisers, derivative-free methods, Bayesian optimisation, and optimal experimental design.

## 7.1 `optimization.gradient` — Gradient-Based Methods

| Class | Description |
|---|---|
| `ScipyOptimizer` | Wrapper around `scipy.optimize.minimize`. Supports L-BFGS-B, SLSQP, trust-constr. Returns `OptimizationResult` with history. |

## 7.2 `optimization.evolutionary` — Derivative-Free Methods

| Class | Description |
|---|---|
| `CMAES` | Covariance Matrix Adaptation Evolution Strategy. Robust for ill-conditioned, non-separable problems up to ∼100 parameters. |

## 7.3 `optimization.bayesian_opt` — Bayesian Optimisation

| Class/Function | Description |
|---|---|
| `BayesianOptimizer` | Sequential BO using GP surrogates from `uq.gp`. Multi-start acquisition optimisation. |
| `expected_improvement(...)` | EI acquisition function |
| `upper_confidence_bound(...)` | UCB acquisition function |

## 7.4 `optimization.oed` — Optimal Experimental Design

| Class | Description |
|---|---|

| | |
|---|---|
| `OptimalExperimentalDesign` | Maximises information criteria (A-, D-, E-optimality) via the Fisher Information Matrix $\mathbf{J}^\top \boldsymbol{\Sigma}^{-1} \mathbf{J}$. |

# Chapter 8

# Project: NSF Domain Mapping

## 8.1 Objective

Learn a diffeomorphism $\varphi : \Omega \to [-1, 1]^2$ that maps an arbitrary 2D domain to the unit square using Neural Spline Flows (NSF). The map is guaranteed to have positive Jacobian determinant by construction, enabling:

1. Discretise and solve PDEs on the simple square domain
2. Map the solution back to the original domain via the exact inverse $\varphi^{-1}$
3. Use the Jacobian determinant for change-of-variables in the PDE

## 8.2 Architecture

The `ConditionalNSF` model consists of:

- **CNN encoder**: Encodes the SDF image of the shape into a latent vector $\mathbf{z} \in \mathbb{R}^{128}$
- **Neural Spline Flow**: 8 coupling layers, each with 16-bin rational quadratic splines, conditioned on $\mathbf{z}$

Key properties of the NSF approach:

- Positive Jacobian determinant guaranteed (no regularisation needed)
- Exact analytical inverse (not iterative)
- Efficient log-determinant computation

## 8.3 Pipeline

The `Main_nsf.py` script runs the full pipeline:

1. **Data generation**: Random shapes → SDF images + harmonic map ground truth
2. **Training**: Supervised learning with interior MSE + boundary MSE + log-det regularisation
3. **Evaluation**: MSE, boundary error, Jacobian statistics, inverse consistency
4. **Visualisation**: Deformed grids, Jacobian fields, predicted vs. ground truth

Listing 8.1: Running the NSF project

```
cd projects/nsf_domain_mapping
python Main_nsf.py
```

## 8.4 Configuration

All hyperparameters are set at the top of `Main_nsf.py`:

| Parameter | Default | Description |
|---|---|---|
| `n_layers` | 8 | Number of NSF coupling layers |
| `n_bins` | 16 | Spline bins per layer |
| `hidden_dim` | 128 | Hidden width in parameter networks |
| `domain_bounds` | $(-1.5, 1.5)$ | Input domain bounds |
| `codomain_bounds` | $(-1.0, 1.0)$ | Target square bounds |
| `learning_rate` | $5 \times 10^{-4}$ | AdamW learning rate |
| `n_epochs` | 300 | Training epochs |
| `batch_size` | 8 | Samples per gradient step |

# Chapter 9

# Compiled Backends

## 9.1 Rust Backend

The Rust backend uses PyO3 and maturin to expose compiled functions to Python. Build with:

```
cd src/simulacrum/_backends/rust
maturin develop --release
```

## 9.2 C++ Backend

The C++ backend uses pybind11 and Eigen. Build with:

```
cd src/simulacrum/_backends/cpp
mkdir build && cd build
cmake .. && make
```

Both backends are optional—the library falls back to pure Python/NumPy/SciPy implementations when they are not available.

# Chapter 10

# Developer Guide

## 10.1 Running Tests

```
pytest tests/                    # All tests
pytest tests/test_materials/     # Specific module
pytest -v --tb=short             # Verbose with short tracebacks
```

## 10.2 Code Style

The project uses `ruff` for linting with a line length of 100 characters:

```
ruff check src/simulacrum/
ruff format src/simulacrum/
```

## 10.3 Type Checking

```
mypy src/simulacrum/
```

Pylance in VS Code provides real-time type checking. Recommended setting: "basic" mode.

## 10.4 Adding a New Project

To start a new research project:

1. Create `projects/my_project/`
2. Add a `launch.json` configuration in `.vscode/` with the correct `cwd`
3. Import from the library: `from simulacrum.geometry import ShapeGenerator`
4. Import locally within the project: `from config import Config`

## 10.5 Git Workflow

Add these to `.gitignore`:

```
__pycache__/
*.pyc
*.egg-info/
```

```
*.vtu
data/
checkpoints*/
outputs*/
.vscode/
```

# Appendix A

# Dependencies

| Package | Group | Purpose |
|---|---|---|
| numpy | core | Array operations |
| scipy | core | Sparse matrices, solvers, sampling |
| meshio | core | Mesh I/O (VTK, gmsh, etc.) |
| pyvista | core | Interactive 3D visualisation |
| vtk | core | VTK data structures |
| matplotlib | core | 2D plotting |
| torch | ml | Neural network training (PyTorch) |
| jax, jaxlib | jax | Automatic differentiation, JIT compilation |
| equinox | jax | Neural networks in JAX |
| optax | jax | Optimisers for JAX |
| diffrax | jax | ODE/SDE solvers in JAX |
| gmsh | meshing | Mesh generation |
| pytest | dev | Testing framework |
| ruff | dev | Linting and formatting |
| mypy | dev | Static type checking |